Information Retrieval and Extraction — Term Project

How Fast Does a News Recommender Go Stale, and How Often Must It Be Rebuilt?
Click-forward evaluation on EB-NeRD (main) and NPR (second dataset)

Noel Alex Jacob (2025201085) Emil Joji (2025201040)
Kspsvln Siddardha Kumar Kavuri (2025201061)

CS4.406 Information Retrieval and Extraction – IIITH – Monsoon 2026

`github.com/N03L-22/IRE_Proj_Draft` (temporary share)

Contents

1 The Question in One Page **1**

2 Words We Use **2**

3 The Scorers We Compare **2**

4 How We Measure It, Step by Step **3**

5 The Data and the Pipeline **4**

6 Results, One Step at a Time **4**

7 The Answer, Put Together **8**

8 What Went Wrong, and What We Changed **8**

9 Limits, Stated Plainly **9**

10 Reproducing This Report **9**

1 The Question in One Page

A news recommender learns from what people clicked in the past. But news articles live for hours, not months. When the recommender is trained once and then left alone (we say it is **frozen**), the articles it learned about are soon gone and new ones arrive that it has never seen. We want to know three things:

1. **How fast** does a frozen recommender get worse?
2. **Which kind of signal** gets worse first: matching words, matching meaning, or learning from who clicked what?
3. **How often** must the recommender be rebuilt to stay good?

The answer, in short (every number is explained later, with its confidence interval):

- **A recommender that learns from clicks goes stale within hours.** One hour after it is frozen it can still say something about 79% of the clicks that follow. After 6 hours that is 19%, after one day 0.9%, and after three days none. New articles simply have no clicks yet.
- **A recommender that reads the article text goes stale slowly, and only through the reader's profile.** If the reader's profile is frozen too, matching words loses about 10% of its score in a week; if the profile is kept fresh, it loses almost nothing.
- **Simply showing the newest articles first beats everything** on the main dataset, and never goes stale. Fusing word and meaning matching helps a little and goes stale in the same way; adding recency to the

fusion with equal weight makes it *worse* than recency alone, and no weight we tried beats recency alone (a sweep, tuned and reported on separate days).

- **How often to rebuild depends on the signal.** The click-based model needs a rebuild at least daily, and hourly on a busy news site. The text-based models can wait days, as long as the reader's profile stays current.

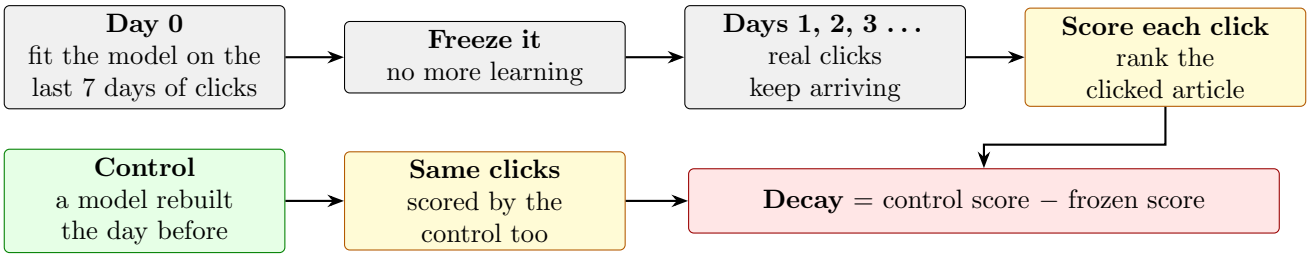


Figure 1: The whole study in one picture. The same real clicks are scored by a frozen model and by a freshly rebuilt one; the difference is the decay.

Read it as: the only thing that differs between the two scores is how old the model is.

2 Words We Use

Word	Meaning
click	one reader opened one article at one moment
pool	the articles that could have been shown at the time of a click: published before it, in the last 48 hours
scorer	a method that gives every article in the pool a score; the highest score is shown first
fit date, frozen	the day the scorer learned from the past 7 days of clicks and then stopped learning
lag	days between the fit date and the click being scored
control	the same scorer, rebuilt the day before the click (lag 1)
decay	control score minus frozen score, on the same clicks; positive means the frozen model is worse
coverage	the share of clicked articles a frozen model is able to score at all
MRR	mean reciprocal rank: if the clicked article is ranked 1st it scores 1, 2nd gives 1/2, 3rd gives 1/3, and so on, averaged over clicks
top-10 hit rate	the share of clicks where the clicked article is among the first 10 (also called recall@10)
confidence interval	$[a, b]$ after a number: if we repeated the study on similar clicks, 95 out of 100 times the number would fall inside

3 The Scorers We Compare

We built the simplest scorers first and added complexity only to see what it buys.

Scorer	What it does	What it cannot do
Random shuffle	puts the pool in random order (the floor)	anything
Recency	newest article first	know what readers like
Popularity	most-clicked articles first	rank an article nobody has clicked yet
Word matching (BM25)	ranks articles by shared words with what the reader read before	see meaning ("car" versus "automobile")
Meaning matching	a frozen multilingual MiniLM model turns each article into 384 numbers; articles close to what the reader read score high	use who else read what
Click history (co-click)	articles that the same people clicked together score high (cosine similarity)	score an article with no clicks or fewer than 5 readers

Two of these are the heart of the story. Word and meaning matching need only the *text* of an article, which exists the moment it is published. Click history needs *clicks*, which exist only after readers have found the article.

4 How We Measure It, Step by Step

4.1 One click, one ranking

Take one real click: reader u opened article a at time t. Build the pool (all articles published at or before t and within the 48 hours before it). The scorer ranks the pool. We record the position of a.

Rank	Article in the pool (invented toy example)	Score
1	Election results	0.91
2	Football transfer	0.74
3	**Storm warning (the clicked article)**	0.66
4	Cooking recipe	0.30
5	Celebrity gossip	0.12

Table 1: A toy pool of five. The clicked article is 3rd, so the reciprocal rank is 1/3, and it is inside the top 10. Real pools hold about 430 articles (EB-NeRD) or 1,500 (NPR).

Averaged over thousands of clicks, this gives the MRR and the top-10 hit rate. Neither needs a person to say what was "relevant": the click itself is the truth, and it happens *after* the model was frozen, so nothing leaks backwards.

4.2 Freezing, waiting, comparing

We fit every scorer on the 7 days of clicks ending on a *fit date*, then score clicks on each later day. We use 14 fit dates (10–23 May 2023) and evaluation days 11–24 May, which gives 105 (fit date, day) cells. For each day the **control** is the model fitted the day before. **Decay = control − frozen** on the same clicks. Each day has 20,000 clicks, chosen by a fixed-seed hash, out of 4–7 million.

4.3 Two ways to treat the reader's profile

Content scorers compare a candidate to what the reader read before. Which clicks count as "before" matters:

- **Frozen profile (regime A):** the clicks up to the fit date. The reader's recent reading is invisible.
- **Live profile (regime B):** the last 50 clicks before the click. Only the article side is frozen.

Comparing A and B separates two kinds of staleness: the model's knowledge of articles and its knowledge of the reader.

4.4 Rules that keep the numbers honest

- **Already-read rule.** If the clicked article is already in the reader's own earlier clicks (15% of clicks on EB-NeRD), it is a re-read, not a discovery; we leave it out and also remove the reader's history from every pool. This rule is the same for every scorer, so comparisons stay fair. (We first forgot it; see Section 8.)
- **Cannot-score rule.** If a scorer cannot score an article, that article sits at the bottom, tied with the others in random order.
- **Fresh versus decayed.** Each clicked article is placed into S0 (published before the fit ended), S1 (published after the fit but before the click day) or S2 (published on the click day). The three parts add up exactly to the decay, so we can say *where* the loss comes from.
- **Confidence intervals.** Every number is a mean with a 95% bootstrap interval ($B = 1000$). The frozen and control scores are resampled with the *same* clicks (paired), which makes small differences visible.

5 The Data and the Pipeline

	EB-NeRD (main)	**NPR** (second)	**Adressa** (turnover only)
Publisher, language	Ekstra Bladet, Danish	Globo, Portuguese	Norwegian news site
Days of clicks	22 (May 2023)	118 (Jan–Apr 2023)	90 (Jan–Mar 2017)
Clicks, users	113.4 M, 791,582	17.6 M, 1,162,802	35.2 M, 3,614,911
Typical age of a clicked article	1.4 h	13.8 h	5.8 h
Evaluated here	14 days × 20,000 clicks	14 days, all impression clicks	no

A *typical clicked article is only 1.4 hours old* on EB-NeRD, against 13.8 hours on NPR. That one number predicts everything that follows. For EB-NeRD we use the click history of the validation users, because its volume and type of click are constant across dates. 27.2% of the 4,468 articles published 4–25 May were never clicked.

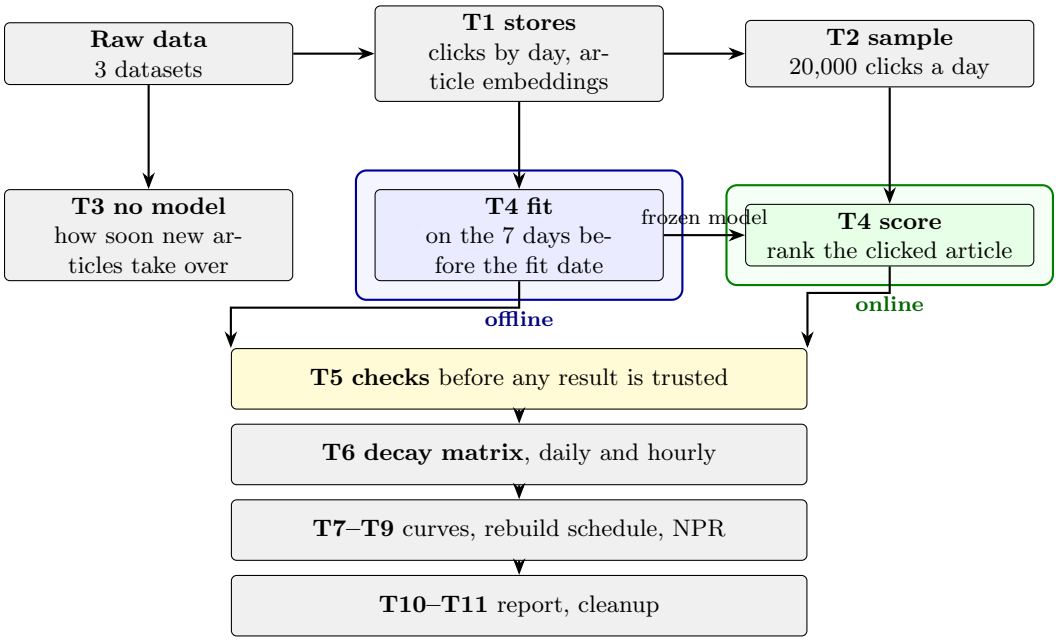


Figure 2: The pipeline. Fitting (offline) and scoring (online) are kept apart, as in a real system. A monitor wraps every unit of work.

Why you can trust the machinery. Every unit of work writes its own file and a line with a hash in a manifest. The T5 gate checks that 1 worker, 2 workers, a reversed order and a resumed run give identical hashes; that recency has a decay of exactly 0 (its frozen and control models are the same function); that the three strata add up to the decay; that a pool allowed to contain post-click articles *fails* (a mutation test); and two planted worlds. In a world with new articles arriving, the checks recover the planted click-history decay (0.0348 ± 0.0049); in a world with none, every decay is within 3.5 standard errors of zero (largest 1.87). 33 unit tests and 8 gate tests pass.

6 Results, One Step at a Time

6.1 Step 1 — How fast do new articles take over? (no model at all)

Before any model: after a model is frozen, what share of the next clicks go to articles that were published *after* the freeze? Such an article is unseen by any model fitted before it.

	1 h	3 h	6 h	12 h	24 h
EB-NeRD	19.5 [14.6, 23.9]	77.2 [72.7, 81.1]	90.8 [88.5, 92.5]	95.4 [94.8, 95.8]	96.2 [95.8, 96.5]
Adressa	6.4 [4.8, 8.3]	24.8 [21.6, 28.1]	48.5 [45.4, 51.5]	73.2 [71.3, 75.1]	87.8 [86.8, 88.7]
NPR	1.0 [0.7, 1.5]	3.2 [2.4, 4.2]	15.5 [11.3, 20.1]	49.8 [41.0, 58.2]	85.6 [80.6, 89.7]

Percent of clicks that go to articles the frozen model never saw (13 fit days for EB-NeRD, 80 for Adressa, 85 for NPR; n at 6 h: 3.46 M, 1.26 M, 72,136 clicks). *Read it as:* on EB-NeRD, 6 hours after a freeze, nine clicks in ten

go to something the model has never seen; on NPR it is fewer than two in ten. The hour of the fit matters too: a fit at midnight stays stale longer than one at 6 in the morning, because few articles are published overnight. EB-NeRD times are read as UTC, which is not confirmed.

6.2 Step 2 — A click-based model, hour by hour

Now a real model. We froze every scorer at fits every 3 hours on 17–21 May and scored the next 24 hourly buckets of 5,000 clicks (40 cells per hour offset). Every hour is also the control of its own bucket. Regime A.

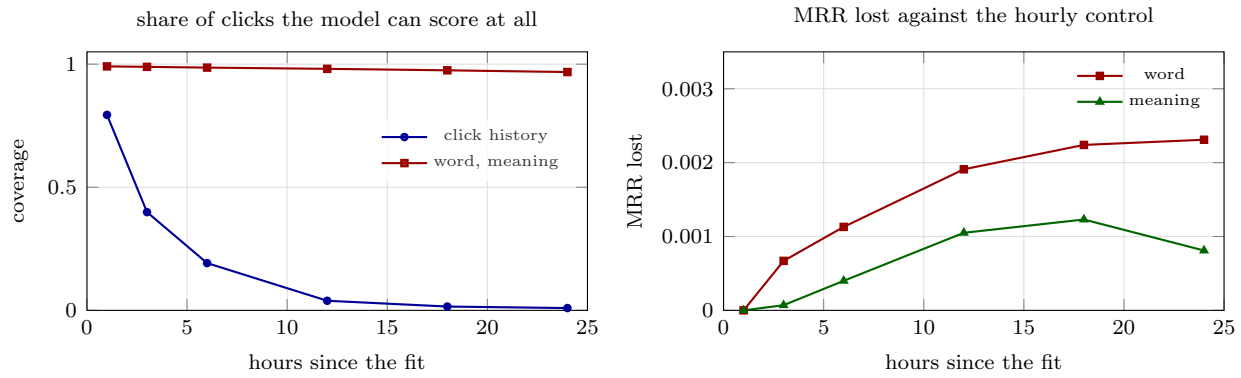


Figure 3: Hourly axis, EB-NeRD, regime A, 40 cells per point. Left: the click-history model collapses within hours. Right: the text models lose very little.

	3 h	6 h	12 h	24 h
Click history coverage	0.399 [0.396, 0.401]	0.192 [0.191, 0.194]	0.039 [0.038, 0.040]	0.0093 [0.0088, 0.0097]
Word, MRR lost	0.00067 [0.00052, 0.00082]	0.00113 [0.00093, 0.00133]	0.00191 [0.00163, 0.00219]	0.00231 [0.00196, 0.00264]
Meaning, MRR lost	0.00007 [-0.00001, 0.00015]	0.00040 [0.00028, 0.00052]	0.00105 [0.00088, 0.00122]	0.00081 [0.00056, 0.00105]

Read it as: the click-based model loses almost all of its reach in a day; the text models lose a few thousandths of an MRR point. Word matching's 24-hour loss, 0.0023 of its 0.0322, is 7%. On the clicks the click-history model can still score at 6 h its top-10 hit rate is 0.045 against 0.104 for the control (a loss of 0.059 [0.055, 0.062]); at 24 h only 0.9% of clicks are scorable and the loss is undetermined (0.009 [-0.009, 0.028]).

6.3 Step 3 — Days, not hours: the profile is what ages

The daily matrix has 14 fit dates and lags of 1 to 13 days. Clicks scored: 236,387 at lag 1, 202,331 at lag 3, 135,583 at lag 7 and 34,479 at lag 13 (there are fewer pairs at long lags; lag 13 is a single cell).

		lag 1	lag 3	lag 7	lag 13
Word, frozen profile (A)	MRR	0.0310 [0.0306, 0.0314]	0.0309 [0.0305, 0.0313]	0.0299 [0.0295, 0.0305]	0.0248 [0.0239, 0.0258]
	lost	0	0.0008 [0.0004, 0.0012]	**0.0030** [0.0024, 0.0036]	**0.0059** [0.0047, 0.0071]
Word, live profile (B)	MRR	0.0325 [0.0321, 0.0329]	0.0331	0.0341 [0.0336, 0.0347]	0.0313
	lost	0	–	0.00013 [0.00007, 0.00019]	–
Meaning, frozen (A)	MRR	0.0250 [0.0247, 0.0253]	0.0247	0.0240 [0.0236, 0.0244]	0.0212 [0.0204, 0.0221]
	lost	0	-0.0002 [-0.0004, 0.0001]	0.0007 [0.0003, 0.0011]	0.0013 [0.0005, 0.0022]
Meaning, live (B)	MRR, lost	0.0259, 0	0.0253, 0	0.0253, 0	0.0233, 0
Random shuffle	MRR	0.0172	0.0177	0.0176	0.0150
Recency	MRR	0.190	0.195	0.196	0.179

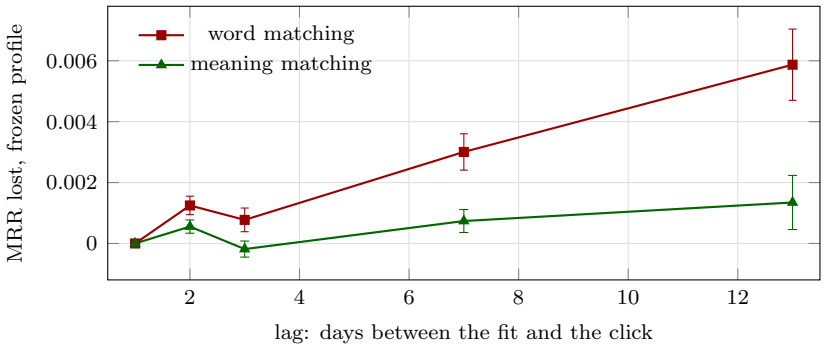


Figure 4: Daily decay of the text scorers with a frozen profile. Bars are the 95% intervals. The loss grows steadily with lag.

Read it as: with a frozen profile, word matching is about 10% worse after a week and 24% worse after nearly two. With a live profile the loss vanishes. Meaning matching with a live profile loses exactly 0, because its article vectors do not depend on the fit and the profile is rebuilt from clicks, so the frozen and control models are the same function. **So the text signal goes stale because the reader's recent interests are missing, not because the articles are old.** Both text scorers are weak (2 to 3 times the shuffle); recency is far ahead (top-10 hit rate 0.515 [0.513, 0.517] against 0.061 for word matching and 0.027 for the shuffle).

Click history and popularity, day by day. Their overall MRR looks *worse than the shuffle* (0.0079 [0.0077, 0.0080] for click history at lag 1 against 0.0172), which would be a misleading way to read it. A target they cannot score is placed below older articles they *can* score, so "cold-start" clicks are penalised. The honest measure is coverage: 12.0% at lag 1, 0.4% at lag 2, zero from lag 3 (the hourly curve above is the same story at finer resolution). On the clicks they *can* score, accuracy is fine; for click history at lag 2 it is 0.139 against 0.141 for the control (0.002 [-0.015, 0.020]), but that is 0.4% of clicks.

6.4 Step 4 — Where does the loss sit?

For word matching with a live profile at lag 7, the total loss is 0.00013. Almost all of it comes from articles published on the click day itself (S2: 0.00012); articles published after the fit but before the day (S1) contribute 0.000006. So the text models lose accuracy on the *newest* articles, not on old ones. For click history at lag 3 the S2 term is −0.0117 and the S1 term +0.0024; their sum is what the cold-start rule produces, which is why coverage is the better measure for this scorer.

6.5 Step 5 — How often should it be rebuilt?

We replayed the same cells with a rebuild every k days, so day d uses the latest rebuild at or before $d-1$. MRR over 14 days, with the cost of the fits:

Rebuild every	1 day (14 fits)	7 days (2 fits)	14 days (1 fit)
Word, frozen profile (A)	0.0310 [0.0306, 0.0314]	0.0298 [0.0295, 0.0302]	0.0274 [0.0270, 0.0277]
Word, live profile (B)	0.0325 [0.0321, 0.0329]	0.0324 [0.0320, 0.0328]	0.0324 [0.0320, 0.0327]
Meaning, A and B	identical at every interval		
Click history (B)	0.0079 [0.0077, 0.0080]	0.0148 [0.0146, 0.0151]	0.0161 [0.0158, 0.0163]
Total fit time	306 s (click history), 29 s (word)	44 s, 4 s	22 s, 2 s

Read it as: with a frozen profile, daily rebuilding of word matching gains 12% over a single rebuild; with a live profile it gains nothing. The apparent rise of click history with a longer interval is a coverage artefact: an older, wider window reaches more of the older articles in the pool, and fewer targets are cold. It is *not* evidence that a stale click-history model is better. One click-history fit costs about 22 s and a peak near 7 GB. A hybrid that falls back to recency for unscorable articles was not built, so these intervals apply to each signal alone.

6.6 Step 6 — A second site: NPR

NPR (Portuguese, a slower news cycle) has 14 fit and evaluation days (10–24 March 2023), all impression clicks, 119,172 at lag 1. The pool is about 1,500 articles. Regime A, lag 1, MRR: popularity 0.162 [0.161, 0.163],

click history 0.105 [0.104, 0.107], word 0.0181 [0.0175, 0.0187], meaning 0.0179 [0.0174, 0.0185], recency 0.0066 [0.0065, 0.0068], shuffle 0.0058 [0.0056, 0.0060]. Top-10 hit rate: popularity 0.550 [0.547, 0.553], click history 0.249 [0.246, 0.251].

- **The ranking of signals is different.** Here popularity and click history are best and recency is no better than the shuffle, because articles live longer (typical age 13.8 h) and the pool is 1,500 articles. A lesson: *which signal to rebuild how often depends on the site.*
- Coverage stays high for click history (97% at lag 1, 96% at lag 7) because new articles take over slowly (only 15.5% unseen at 6 h).
- Word matching with a frozen profile loses 0.0012 [0.0010, 0.0015] at lag 3 and 0.0024 [0.0020, 0.0029] at lag 7 (14% of its 0.0167); meaning loses 0.0022 [0.0017, 0.0027] at lag 7. The text-model story repeats.
- **Unexplained.** Popularity gets *better* with an older fit (its "decay" at lag 3 is −0.112 [-0.113, -0.111] MRR). We have not found the cause and report it only as a measurement.

6.7 Step 7 — A fusion of word and meaning matching

What if one model used both text signals? We built two fusions, with *reciprocal rank fusion*: each signal ranks the pool, and an article scores $\sum 1/(60+\text{rank})$ over the signals, with equal weights and nothing tuned (tuning on the evaluation clicks would leak). Tied scores share a rank, so a tie never favours old or new articles. **Content fusion** uses word and meaning; **content plus recency** adds the newest-first ranking. Same 14 fit dates, same 236,387 clicks at lag 1.

		lag 1	lag 3	lag 7	lag 13
Content fusion, frozen profile (A)	MRR	0.0329 [0.0325, 0.0333]	0.0337	0.0330 [0.0325, 0.0336]	0.0254 [0.0244, 0.0263]
	lost	0	-0.0001 [-0.0006, 0.0002]	**0.0023** [0.0017, 0.0028]	**0.0053** [0.0041, 0.0065]
Content fusion, live (B)	MRR, lost	0.0345, 0	0.0350, 0.00002	0.0366, 0.00005	0.0319, 0.0002
Content + recency, frozen (A)	MRR	0.129 [0.128, 0.130]	0.135	0.144 [0.143, 0.146]	0.148 [0.145, 0.150]
	lost	0	-0.0039 [-0.0046, -0.0031]	-0.0080 [-0.0093, -0.0069]	-0.0185 [-0.0211, -0.0161]
Content + recency, live (B)	MRR, lost	0.1286, 0	0.1305, 0.00001	0.1346, 0.00006	0.1272, 0.0002

With a live profile the loss stays under 0.0003 at every lag (intervals in `results/fusion/rq1_rq2/decay_by_lag.csv`). Top-10 hit rate, live profile, lag 1: content fusion 0.069 [0.068, 0.070] against 0.061 for word and 0.047 for meaning alone; content plus recency 0.266 [0.264, 0.268] against 0.515 for recency alone.

- **Content fusion is a little better than either part and goes stale the same way:** 0.0023 lost by lag 7 with a frozen profile (7% of its level), almost nothing with a live one.
- **Adding recency with equal weight makes things worse than recency alone** (MRR 0.129 against 0.190; hit rate 0.27 against 0.52). Two weak voices outvote one strong one. A weight sweep follows below.
- **With recency in, the frozen profile scores *better* than the fresh one** (a "loss" of −0.0185 at lag 13). We have not tested why. A possible reason is that a stale profile makes the text part less confident, so recency dominates the fused ranking, and recency is the stronger signal here. This is a guess, not a finding.

What if the weights were wrong? A sweep. We varied the recency weight (0.1, 0.25, 0.5, 1, 2), the fusion constant K (10, 60, 200) and the word weight (0.5, 1, 2), one at a time (14 settings). To avoid tuning on the clicks we report, the 14 evaluation days were split into two interleaved folds (alternate days). On one fold a setting's score was its MRR averaged over lags 1, 2, 3, 5, 7 and both profile regimes (rule fixed beforehand, decision D29); the best setting was then reported on the *other* fold only, and the folds were swapped.

Reported on days of fold	MRR, live profile, lag 1	top-10 hit rate, lag 1	MRR lost, lag 7, frozen / live profile	clicks at lag 1
Chosen fusion (recency weight 2), fold O	0.1868 [0.1854, 0.1883]	0.433 [0.430, 0.436]	-0.0008 [-0.0025, 0.0009] / -0.00003	118,610
Recency alone, fold O	0.1910 [0.1896, 0.1924]	0.518 [0.515, 0.521]	0	118,610
Chosen fusion (recency weight 2), fold E	0.1898 [0.1883, 0.1913]	0.451 [0.449, 0.454]	0.0035 [0.0017, 0.0051] / 0.00013	117,777
Recency alone, fold E	0.1899 [0.1886, 0.1913]	0.512 [0.509, 0.515]	0	117,777

- **More recency weight is always better.** On the tuning fold E the MRR scores rise steadily with the recency weight: 0.046 at 0.1, 0.058 at 0.25, 0.082 at 0.5, 0.134 at 1 and 0.193 at 2 (tuning numbers, not results). Both folds chose the largest weight we tried.
- **The best fusion only matches recency alone, and is worse on the top-10 hit rate** (0.43–0.45 against 0.51–0.52). So *no setting we tried beats ranking by recency alone.* The optimum is at the edge of our grid, and a larger weight would approach recency alone.
- **The frozen-profile loss is not stable across folds** (0.0035 on one, −0.0008 on the other, intervals overlapping zero in one), so we draw no conclusion about its sign for the best fusion. With a live profile it is about 0.

- The earlier guess that a stale profile helps because recency dominates is neither confirmed nor ruled out.

6.8 Step 8 — Checks on our own results

Does removing never-clicked articles change the story? 27.2% of the articles in the pool were never clicked in the whole corpus. A recommender cannot know that in advance (it uses future clicks), so this is an oracle ceiling, not a realistic setting. We removed them from every pool, with identical clicks (17,037 on the first fit in both runs). Regime B, lag 1, top-10 hit rate:

	Recency	Shuffle	Meaning / Word
All articles in the pool	0.515 [0.513, 0.517]	0.027 [0.026, 0.027]	0.047 / 0.061
Never-clicked removed	0.634 [0.632, 0.636]	0.040 [0.039, 0.041]	0.062 [0.061, 0.063] / 0.063 [0.062, 0.064]

Every level rises, but the decay does not change: word matching, live profile, MRR lost at lag 7 is 0.00013 in the main run and 0.00010 [0.00003, 0.00016] here; meaning and recency stay at 0. *So the pool noise lifts every score equally and the staleness result stands.*

Does decay follow the amount of news per day? (exploratory) At lag 3 over 12 days, Spearman ρ between word MRR lost and articles published that day (152–266) is -0.42 [-0.87, 0.24]; against clicks that day (4.2–6.8 M) -0.52 [-0.98, 0.20]. For click history $+0.43$ [-0.14, 0.81] and -0.20 [-0.77, 0.55]. Every interval includes 0: **no relation is detectable** with 12 days from one site. This needs a longer span or more sites.

7 The Answer, Put Together

Signal	**Why it goes stale**	**How often to rebuild (EB-NeRD)**
Recency	it does not	never
Click history	new articles have no clicks: coverage 79% at 1 h, 19% at 6 h, 0.9% at 24 h	at least daily; hourly on a busy site (one fit: 22 s, about 7 GB)
Popularity	the same, because it counts clicks	the same as click history
Word matching	the reader's profile ages (10% lost in 7 days, 24% in 13); articles hardly matter	keep the profile live; the word index itself can wait about a week
Meaning matching	only a small profile effect (0.0007 [0.0003, 0.0011] by lag 7); article vectors never change	never for decay; refresh the profile

Why the same models decay differently on another site. What predicts the decay is how quickly articles take over (Section 6.1): 91% unseen at 6 h on EB-NeRD, 49% on Adressa, 16% on NPR. A site with slower turnover can rebuild less often.

8 What Went Wrong, and What We Changed

- **We scored articles the reader had already read (the main error).** Our first full run included re-reads. Our own check found that 15% of the clicked articles were already in the reader's history, and that already-read articles outranked the target. This inflated word matching (MRR 0.0573 with them, 0.0143 without) and made a fresher profile look worse. We deleted every result of that run and reran everything under the already-read rule.
- **Two hourly units were marked complete but 19 of their files were missing.** Found when the analysis failed; recomputed. The cause is unknown. A manifest-versus-files check belongs in the final cleanup.
- **We said word matching "barely goes stale".** That was true only for the live profile. With a frozen profile it loses 10% in a week (Step 3). Corrected here and in the mid-submission report.
- **Memory.** Click history needs about 7 GB per fit, not the 1.5 GB we planned, so only two workers run at once.
- **The corpus.** Our first plan assumed one uniform click corpus. EB-NeRD is not uniform, so we chose click-forward evaluation on one cohort.
- **Duplicate rows** in the first stores (107,633 in NPR, 7,693 in Adressa), found by counting and rebuilt.

The full record (decisions D1–D28, findings P1–P48, dated errata, session log) is in `log.md`.

9 Limits, Stated Plainly

- **One main site, one month, one cohort** (the validation users), 14 fit days. NPR is a second site; Adressa has the turnover measure only.
- **The negatives are the other pool articles**, not real impression slates, so levels are low and overall MRR is penalised by cold start. We report coverage next to it. The team still has to pick the headline metric.
- **The 48-hour pool window, the sample size and the seed were not varied.** Sensitivity runs were planned and not done.
- **EB-NeRD times are assumed to be UTC.** Hour labels may be shifted by 1–2 hours.
- **Decay is measured against a model rebuilt one day (or one hour) earlier.** Faster rebuilds were not tested. The bootstrap treats days as given, so intervals do not cover day-to-day variation.
- **The fusion sweep is one setting at a time on a grid whose best point is at the edge** (recency weight 2), with 7 tuning days per fold. A fallback for unscorable articles was not built, and there is no position-based baseline.
- **Popularity improving with an older fit on NPR is unexplained.**
- **The never-clicked check and the volume check are not realistic settings or findings**; they are labelled as such.

10 Reproducing This Report

From `code/`, with the project's Python environment (`<root>` is the project folder):

- **Daily matrix:** `decay.py <root> --fits 2023-05-10 2023-05-23 --last 2023-05-24`
- **Hourly:** `decayh.py <root> --first 2023-05-17 --last 2023-05-22 --fit-days 2023-05-17 2023-05-21 --step 3`
- **Tables with intervals:** `analyze.py <root>` and `analyze_hourly.py <root>`
- **NPR:** the same with `--name npr --out results/npr/decay_matrix`
- **Never-clicked check:** `decay.py` with `--clicked-only --out results/sens_clicked_only/decay_matrix`
- **Volume check and gate:** `volume_check.py <root>`, `test_t5.py`

Data stays under `data/` (not in the repository). Run logs and the gate record are in `results/metrics/`. The Adressa tarball was never extracted.